

Session與Cookie

- **11-1:網頁狀態保存種類**
 網頁狀態保存種類.....11-2
- **11-2:Session**
 Session 是什麼.....11-3
 Session 的特色.....11-4
 使用Session.....11-5
- **11-3:Cookie**
 Cookie 是什麼.....11-6
 寫入、讀取與刪除Cookie.....11-7
 CookieOptions 常用安全設定.....11-8
- **11-4: 比較Session與Cookie**
 Cookie 與 Session 的關係.....11-9
 Session 是否被 Cookie 取代？.....11-10
 Session vs Cookie.....11-12

Module 11

網頁狀態保存種類

	Application	Static	Session	Cookie	Database
Server端技術	v	v	v		v
Client端技術				v	
存於磁碟				v	v
存於記憶體	v	v	v		
支援字串	v	v	v	v	v
支援物件		v	v		
瀏覽器獨立			v	v	
使用者共用性	v	v			v
生命週期	應用程式結束	應用程式結束	預設閒置20分鐘，可調整	預設瀏覽器關閉，可調整	永久

Session 是什麼

- **Session 是一種 Server 端狀態管理機制。**
- **用來在多個 Request 之間保存使用者相關資料。**
- **瀏覽器只保存 Session Id (通常存在 Cookie)。**
- **實際資料存放於 伺服器端 (Memory / Redis / SQL)。**

Session 的特色

- 資料存在 **Server** 端，安全性較高。
- 適合儲存：
 - 登入狀態
 - 操作流程中的暫存資料
 - 購物車、**Wizard** 流程
- 須注意：
 - 記憶體使用量
 - 多台主機需額外處理 (**Sticky Session** / 分散式 **Session**)

使用Session

- **Program.cs :**

```
app.UseSession();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

- **Controller :**

```
public IActionResult SetSession()
{
    HttpContext.Session.SetString("UserName", "資展工程師");
    HttpContext.Session.SetInt32("Counter", 1);
    return Content("Session 已寫入");
}

public IActionResult GetSession()
{
    var userName = HttpContext.Session.GetString("UserName");
    var counter = HttpContext.Session.GetInt32("Counter");

    return Content($"UserName={userName}, Counter={counter}");
}

public IActionResult RemoveSession()
{
    HttpContext.Session.Remove("UserName"); // 刪除單一 key
    return Content("Session key 已移除");
}

public IActionResult ClearSession()
{
    HttpContext.Session.Clear(); // 清空全部
    return Content("Session 已清空");
}
```

Cookie 是什麼

- **Cookie** 是瀏覽器儲存在用戶端的一小段資料 (**Key/Value**) 。
- 每次發送 **HTTP Request** 時，瀏覽器會自動把符合條件的 **Cookie** 帶回伺服器。
- 常見用途：登入狀態、偏好設定、追蹤識別碼。
- **Cookie** 的特性與限制：
 - 容量小：單一 **Cookie** 約 **4KB** (不同瀏覽器有差異)
 - 會跟著請求送出：設定不當會增加流量與風險
 - 可設定有效範圍：**Domain / Path / Expires(Max-Age)**

寫入、讀取與刪除Cookie

- 寫入Cookie：

```
public IActionResult SetCookie()
{
    Response.Cookies.Append("theme", "dark", new CookieOptions
    {
        Expires = DateTimeOffset.UtcNow.AddDays(7),
        HttpOnly = true,          // 防 JS 讀取
        Secure = true,            // 只在 HTTPS 傳送
        SameSite = SameSiteMode.Lax
    });

    return Ok("Cookie set");
}
```

- 讀取與刪除Cookie：

```
public IActionResult GetCookie()
{
    var theme = Request.Cookies["theme"];
    return Ok(theme ?? "no cookie");
}
```

```
public IActionResult DeleteCookie()
{
    Response.Cookies.Delete("theme");
    return Ok("Cookie deleted");
}
```

CookieOptions 常用安全設定

- **HttpOnly**
 - **true** : JavaScript 讀不到 (降低 XSS 竊取 Cookie 風險)
- **Secure**
 - **true** : 只在 HTTPS 傳送 (正式環境建議必開)
- **SameSite**
 - **Strict** : 最嚴格，幾乎不允許跨站帶 Cookie (防 CSRF 強，但有時影響登入流程)
 - **Lax** : 折衷，常用預設，對一般導頁較友善
 - **None** : 允許跨站帶 Cookie (必須搭配 **Secure=true**)
- **Expires / MaxAge**
 - 控制 Cookie 何時失效 (會話 Cookie vs 持久 Cookie)

設定 Cookie 與 Session 的關係

- **Cookie :**
 - 儲存在 **Client** 端
 - 容量小 (約 4KB)
- **Session :**
 - 儲存在 **Server** 端
 - 通常透過 **Cookie** 傳遞 **Session Id**
- **Session 並不是不用 Cookie , 而是「用 Cookie 當鑰匙」。**

Session 是否被 Cookie 取代？ -1

- **Session 並沒有被 Cookie 取代，但使用情境已經明顯改變。**
- **為什麼傳統 Session 使用率下降？**
 - 雲端與分散式架構盛行
 - **RESTful / API-first 設計成主流**
 - 前後端分離（**SPA / Mobile App**）
 - 系統偏向 **Stateless（無狀態）Session**：
- **傳統 Server-side Session 維運成本高**

Session 是否被 Cookie 取代？-2

- **Session** 沒有消失，但不再是所有系統的預設解法。
- 是否使用 **Session**，取決於：
 - 架構型態
 - 是否分散式
 - 是否前後端分離
- 在現代雲端與 **API** 架構中，系統多採用無狀態設計，以 **Token**（如 **JWT**）搭配 **Cookie** 或 **Header** 取代傳統 **Session**；但在 **Server-side MVC** 網站中，**Session** 仍然是合理且實用的選擇。

Session vs Cookie

情境	傳統作法	現代趨勢
MVC Server-side 網站	Session	仍常用
登入驗證	Session	JWT + Cookie / Header
API / 微服務	不適合	Stateless + Token
使用者偏好設定	Session	Cookie
高併發系統	Session 困難	Token-based